

hardware and software. Let's take a look at the software components one by one.

Kernel of the OS

The literal meaning of the word kernel is 'core'. The kernel is responsible for handling pretty much everything else on the system. The kernel knows how to use the processor in the SoC above everything else. It knows the codes to send to the processor. Therefore, you can't use an OS written for ARM microarchitecture to work on an Intel microarchitecture because the kernel uses different instruction sets. In fact, the processor would not be able to even load the OS when the system boots! The kernel can be monolithic, microkernel or a hybrid one. Since SoCs of today have become quite sophisticated, the kernel of OSes they run are big and sophisticated too.

Drivers

The kernel knows how to control the microprocessor in the system. However, it cannot always know how to control every single component connected to it. For example, if there is a Bluetooth module in the SoC, then depending on the make and model, the unit may or may not have certain abilities (think Bluetooth profiles) and support a specific version of the protocol (e.g. Bluetooth 4.0).

For making sure the OS can work with multiple types of hardware, drivers are created which can work with the hardware in the SoC. Drivers call certain APIs in the OS and the OS can call the registered methods in driver to control the hardware back. Such drivers are device drivers. There can be other drivers too. One such example would be a file-system driver – when you attach an NTFS formatted USB to your mobile device, the OS while with being able to talk to the hardware must also know how to interpret what's on the disk so that it can present it to you in the file explorer. Drivers execute in privileged mode but on a higher ring than kernel so that they can be controlled by the kernel. This design allows the kernel to handle crashes in drivers gracefully.

Consider this: if you plug a USB device into your mobile via OTG and the file system driver crashes then your phone would crash too if the problem went undetected by the kernel. However, if the OS could detect the crash in the driver, it could just stop the driver (and any process that was using its functions) and show a message to user saying that the USB could not be read and silently re-load the driver in the background.